

```
>>> introduction('Alice')
Hello, Alice
>>> introduction('Hiro', 'Ohiyo gozaimasu')
Ohiyo gozaimasu, Hiro
```

When the `introduction()` function is called without a second argument, it uses the string `'Hello'` by default. Note that parameters with default arguments must always come after parameters without default arguments.

Recall from Chapter 8 that you should avoid using a mutable object, such as an empty list `[]` or empty dictionary `{}`, as the default value. “Don’t Use Mutable Values for Default Arguments” on page 143 explains the problem that this approach causes and its solution.

Using * and ** to Pass Arguments to Functions

You can use the `*` and `**` syntax (often pronounced as *star* and *star star*) to pass groups of arguments to functions separately. The `*` syntax allows you to pass in the items in an iterable object (such as a list or tuple). The `**` syntax allows you to pass in the key-value pairs in a mapping object (such as a dictionary) as individual arguments.

For example, the `print()` function can take multiple arguments. It places a space in between them by default, as the following code shows:

```
>>> print('cat', 'dog', 'moose')
cat dog moose
```

These arguments are called *positional arguments*, because their position in the function call determines which argument is assigned to which parameter. But if you stored these strings in a list and tried to pass the list, the `print()` function would think you were trying to print the list as a single value:

```
>>> args = ['cat', 'dog', 'moose']
>>> print(args)
['cat', 'dog', 'moose']
```

Passing the list to `print()` displays the list, including brackets, quotes, and comma characters.